

Experimento 1 — Validação consolidada da Gold

Implementação e testes das contagens de nulos/vazios em fato_internacao

10 de setembro de 2026

Sumário

1	Resultado e escopo	1
2	Contexto e baseline observado	1
3	Código encontrado e regras preservadas	2
4	Implementação consolidada	3
5	Instrumentação de tempo	3
6	Arquivos e limites da alteração	4
7	Testes automatizados criados	4
8	Resultados das validações	4
9	Revisão do diff	5
10	Riscos e validações pendentes	5
11	Comandos para reprodução e experimento posterior	6
11.1	Testes sintéticos	6
11.2	Verificações existentes	6
11.3	Execução real posterior, após revisão	6
12	Situação de entrega	6

1 Resultado e escopo

Foi implementado somente o primeiro experimento identificado na análise operacional: consolidar as contagens de nulos/vazios do bloco “Validando campos importantes...” de `src/gold/fato_internacao.py`.

As 12 contagens separadas foram substituídas por **uma agregação Spark com uma action**, mantendo as regras, a lista de colunas, os logs individuais e a reprovação na primeira coluna inválida. Foi acrescentada instrumentação de tempo apenas ao bloco.

Os testes sintéticos confirmaram equivalência lógica. O ganho de desempenho sobre os dados reais ainda não foi medido.

Não foram executados a DAG completa nem o job Gold sobre os dados reais. Não houve sobrescrita de datasets, alteração de permissões, commit ou push. Este relatório e sua versão PDF documentam o trabalho já realizado; não introduzem novas mudanças na implementação.

2 Contexto e baseline observado

A análise anterior identificou a task `gold.gold_fato_internacao` no caminho crítico da DAG `tcc_pipeline`, com 5 TaskGroups e 24 tasks.

Medição anterior	Valor
Duração da DAG completa	41m46,422s
Duração da task Gold fato internação	617,946s
Bloco de validação de campos importantes	273,001s
Participação do bloco na task	44,18%
Participação do bloco na DAG	10,89%

Esses valores pertencem à run `**manual__2026-09-10T01:37:19.578492+00:00**`, anterior à otimização. Eles não são resultados da nova implementação.

Métrica	Antes	Depois
Actions no bloco aprovado	12	1
Validações preservadas	12	12
Resultado esperado	Igual	Igual nos testes sintéticos
Tempo baseline observado	273,001s	A medir

Na versão anterior, uma reprovação antecipada podia executar menos de 12 actions. Agora, todas as métricas são calculadas juntas antes da sequência de logs e validações.

3 Código encontrado e regras preservadas

A versão atual do script foi lida antes da alteração. O bloco anterior executava, dentro do loop de campos:

```
quantidade_nulos = (
    df_gold
    .filter(
        col(campo).isNull()
        | (trim(col(campo).cast("string")) == "")
    )
    .count()
)
```

A regra era a mesma para todas as 12 colunas, nesta ordem:

Ordem	Coluna
1	ano
2	mes
3	CD_MUN
4	NM_MUN
5	SIGLA_UF
6	codigo_cid
7	descricao_cid
8	codigo_categoria
9	descricao_categoria
10	descricao_grupo
11	codigo_capitulo
12	descricao_capitulo

Foi preservada exatamente a condição **NULL ou string vazia após cast para string e trim**. O cast continua sendo usado apenas na expressão de validação: não modifica o schema do DataFrame.

O comportamento de trim não foi ampliado. Nos testes, espaços comuns isolados são inválidos, enquanto tabulação e quebra de linha isoladas mantêm o comportamento anterior do Spark. Valores numéricos zero e negativos não passam a ser inválidos por esta regra.

O critério de reprovação continua sendo **quantidade_nulos > 0**. O log por coluna permanece:

```
{campo}: {quantidade_nulos} nulos/vazios
```

A exceção permanece:

```
raise ValueError(  
    f"O campo {campo} "  
    "possui valores nulos/vazios."  
)
```

A ordem da lista é mantida. Ao encontrar a primeira métrica inválida, o loop imprime essa métrica e lança o mesmo ValueError, sem imprimir as colunas posteriores.

4 Implementação consolidada

Foram adicionados os imports de perf_counter, da biblioteca padrão, e sum do PySpark com o alias spark_sum. As demais funções necessárias já estavam importadas.

O cálculo passou a ser:

```
inicio_validacao_campos = perf_counter()  
  
metricas_campos_importantes = df_gold.agg(  
    *[  
        coalesce(  
            spark_sum(  
                when(  
                    col(campo).isNull()  
                    | (trim(col(campo).cast("string")) == ""),  
                    1,  
                ).otherwise(0)  
            ),  
            lit(0),  
        ).alias(campo)  
        for campo in campos_importantes  
    ]  
) .first()
```

A lista constrói 12 expressões nativas de agregação; ela não executa uma action por coluna. A única action explícita para obter as métricas é first(), que retorna uma Row pequena com as 12 contagens.

O coalesce com zero preserva o comportamento de filter(...).count() para um DataFrame vazio: sum retornaria NULL nesse caso, enquanto a lógica anterior retornava zero.

O loop existente passa a consultar a Row:

```
for campo in campos_importantes:  
    quantidade_nulos = metricas_campos_importantes[campo]  
    # Logs e ValueError existentes permanecem.
```

Não foram adicionados UDFs, pandas UDFs, RDDs, cache ou persist. Uma action não implica obrigatoriamente um único job ou estágio interno do Spark; a redução comprovada é de chamadas de action no bloco.

5 Instrumentação de tempo

O cronômetro inicia imediatamente antes da construção e execução da agregação. Após a aprovação das 12 validações, é emitido:

```
print(  
    "[PERFORMANCE] Validação consolidada de campos importantes: "  
    f"{perf_counter() - inicio_validacao_campos:.3f} segundos"  
)
```

O tempo inclui o cálculo agregado e o loop de logs/validações. Não mede o pipeline inteiro.

Se uma coluna reprovar, o ValueError interrompe o bloco antes do log de performance. Isso mantém a sequência funcional de reprovação; não foi introduzido tratamento adicional de exceções.

6 Arquivos e limites da alteração

Arquivo	Ação
src/gold/fato_internacao.py	Modificado: imports, agregação, leitura da Row e cronômetro
tests/test_fato_internacao_metricas.py	Criado: seis testes sintéticos e verificação estática
docs/Relatorio_Experimento_1_Validacao_Gold.md	Criado nesta documentação
docs/Relatorio_Experimento_1_Validacao_Gold.pdf	Criado nesta documentação

O cálculo de métricas não reatribui nem transforma `df_gold`. Joins, filtros de negócio, granularidade, paths, escrita, schema, `partitionBy`, `repartition`, `broadcast`, `shuffle`, AQE e `SparkSession` de produção permaneceram intactos.

Não foram modificados Airflow, DAG, DQs, Silver, Bronze ou outras Gold. As alterações preexistentes no workspace foram preservadas.

7 Testes automatizados criados

O script Gold executa processamento no topo do módulo. Para evitar sua execução real, o teste lê o arquivo, extrai o bloco 18 pelos seus marcadores e compila somente esse bloco e os imports usando AST.

Assim, o teste executa **a agregação real de produção**, sem manter uma cópia alternativa da nova lógica. A referência anterior é calculada independentemente com `filter(...).count()` para todas as colunas.

Os DataFrames são pequenos e sintéticos. O schema principal usa ano inteiro, mes longo e os demais campos textuais; há também um caso com todas as colunas textuais para verificar o cast. Não se lê o conjunto real de 8,4 milhões de registros.

Teste	Verificação
Valores válidos e trim	Valores válidos, espaços ao redor de texto, zero, negativos, tabulação e quebra de linha
Nulos, vazios e múltiplos inválidos	Todas as 12 métricas; contagens distintas por coluna para detectar aliases trocados
DataFrame vazio	Todas as métricas iguais a zero
Primeiro erro após colunas válidas	Ordem dos logs e <code>ValueError</code> da primeira coluna inválida
Colunas textuais	Cast e tratamento de strings vazias/espaços em todos os campos
Uma agregação e uma action	AST com um agg e um first, sem filter, count, collect, cache ou persist no bloco

Também são verificados: lista e ordem das colunas, igualdade das contagens antigas e novas, identidade e schema do DataFrame, logs funcionais, mensagem da exceção e formato da instrumentação.

8 Resultados das validações

Os resultados abaixo foram obtidos durante a implementação. A criação deste relatório não repetiu o pipeline nem o experimento real.

Validação	Resultado
Seis testes novos	OK — 6 testes em 13,637s
Testes existentes de orquestração Bronze	OK — 5 testes em 2,929s
Compilação da Gold e do novo teste	Aprovada
Imports de produção isolados	Aprovados
airflow dags list-import-errors	No data found
Quantidade de TaskGroups	5
Quantidade de tasks	24

Validação	Resultado
Dependências da DAG	Testes existentes aprovados
Integridade do arquivo da DAG	SHA-256 idêntico ao anterior
Busca e análise estática do bloco	Uma agregação e uma action
git diff -check	Aprovado

Os testes existentes incluem dependências, grupos e tasks da DAG, gates e wrappers Bronze. As verificações de aquisição usam mocks ou checam arquivos existentes sem download; extrações sintéticas usam diretório temporário.

A primeira tentativa dos testes novos pelo Python direto carregou PySpark 4.2.0 em modo Spark Connect e falhou em setUpClass por ausência de URL remota. Nenhum teste foi executado nessa tentativa. O comando foi ajustado para o **spark-submit usado pela DAG**, que executou Spark **3.5.0**, com Java 17, e aprovou os seis testes. O ambiente não foi alterado.

Foram emitidos avisos do ambiente, incluindo ResourceWarning de sockets durante os testes Spark, sem reprovação dos testes.

O hash preservado de dags/tcc_pipeline.py foi:

7BEE4CF50BBE3422F1B0030230D5B1266AC39AF926DF65D8F16ACE573FC25738

9 Revisão do diff

O git diff -stat dos arquivos rastreados, reconferido para este relatório, mostrou:

```
dags/tcc_pipeline.py          | 66 ++++++
src/gold/fato_internacao.py | 45 ++++++-----
2 files changed, 97 insertions(+), 14 deletions(-)
```

As **66 inserções na DAG eram preexistentes** e não pertencem ao experimento. Nesta implementação, a Gold teve **31 inserções e 14 remoções**. O novo teste possui **148 linhas** e, por estar não rastreado, não aparece no git diff -stat acima. O mesmo vale para arquivos de documentação não rastreados.

O diff de produção contém somente:

1. Import de perf_counter.
2. Import de sum como spark_sum.
3. Cálculo conjunto das métricas antes do loop.
4. Substituição de filter(...).count() pela consulta à Row.
5. Log de performance após a validação.

O loop de logs e a exceção foram preservados. O diff integral dos arquivos rastreados foi revisado na implementação; o teste criado também foi inspecionado.

10 Riscos e validações pendentes

O risco funcional é baixo para as condições testadas, pois as mesmas expressões e contagens foram confirmadas em DataFrames sintéticos. Há limites explícitos:

- Todas as métricas agora são avaliadas antes da primeira reprovação. O ValueError de negócio mantém sua ordem, mas eventuais falhas internas do Spark em outras expressões podem surgir durante a agregação conjunta.
- As 12 métricas retornam uma Row pequena; isso não representa coleta do dataset completo para o driver.
- A mudança não garante que a duração do bloco caia na mesma proporção da redução de actions.
- Os testes verificam equivalência lógica e preservação do DataFrame no bloco, não igualdade de arquivos Gold reais produzidos após uma execução completa.
- Ainda é necessário medir tempo e recursos no dataset real e confirmar aprovação do DQ da fato.
- O teste depende dos marcadores dos blocos 18 e 19. Se eles forem renomeados, o teste precisará acompanhar essa alteração.

Não há resultado novo de desempenho para comparar aos 273,001s. Não se afirma redução comprovada dos 41m46s da DAG.

11 Comandos para reprodução e experimento posterior

11.1 Testes sintéticos

Comando utilizado no worker, a partir da raiz do repositório:

```
docker compose -f docker-compose.airflow.yaml exec -T -w /home/jovyan/work airflow-worker  
↳ spark-submit --master "local[2]" --conf spark.ui.enabled=false  
↳ tests/test_fato_internacao_metricas.py -v
```

11.2 Verificações existentes

```
docker compose -f docker-compose.airflow.yaml exec -T -w /home/jovyan/work airflow-worker  
↳ python -B -m unittest discover -s tests -p test_bronze_orchestration.py -v  
  
docker compose -f docker-compose.airflow.yaml exec -T airflow-scheduler airflow dags  
↳ list-import-errors
```

11.3 Execução real posterior, após revisão

O comando abaixo preserva as configurações da task identificadas na DAG. **Ele executa a Gold real e sobrescreve sua saída conforme a escrita atual do script. Não foi executado nesta tarefa.** A execução direta não aciona automaticamente o DQ posterior do Airflow.

```
docker compose -f docker-compose.airflow.yaml exec -T airflow-worker spark-submit --master  
↳ "local[2]" --conf spark.ui.enabled=false --conf spark.sql.autoBroadcastJoinThreshold=-1  
↳ --name tcc-gold-fato-internacao --deploy-mode client  
↳ /home/jovyan/work/src/gold/fato_internacao.py
```

Após aprovação para o experimento, registrar o log de performance e a duração total da task. O tempo externo do comando spark-submit não é rigorosamente igual ao tempo da task Airflow, que inclui seus próprios custos de inicialização e encerramento.

Para comparação mais confiável, manter as mesmas entradas e condições de execução, comparar múltiplas observações e validar os resultados e o DQ. O valor de 273,001s é uma referência observada em uma run, não uma distribuição estatística.

12 Situação de entrega

Implementação localizada concluída e testes sintéticos aprovados. As 12 regras e suas mensagens foram preservadas, com uma agregação e uma action no bloco. A DAG permanece com 5 TaskGroups e 24 tasks.

Pendente: revisão da implementação, execução real autorizada, medição de desempenho e validação funcional sobre os dados completos.